

BrainPilot Agent 构建学习手册

面向缺少 Agent 构建经验、希望通过 Codex (vibe coding) 复刻并进一步扩展 BrainPilot 的开发者 基于当前工作区源码分析，版本：**0.1.1**，分析日期：2026-07-25

阅读目标

读完并完成练习后，你应该能够：

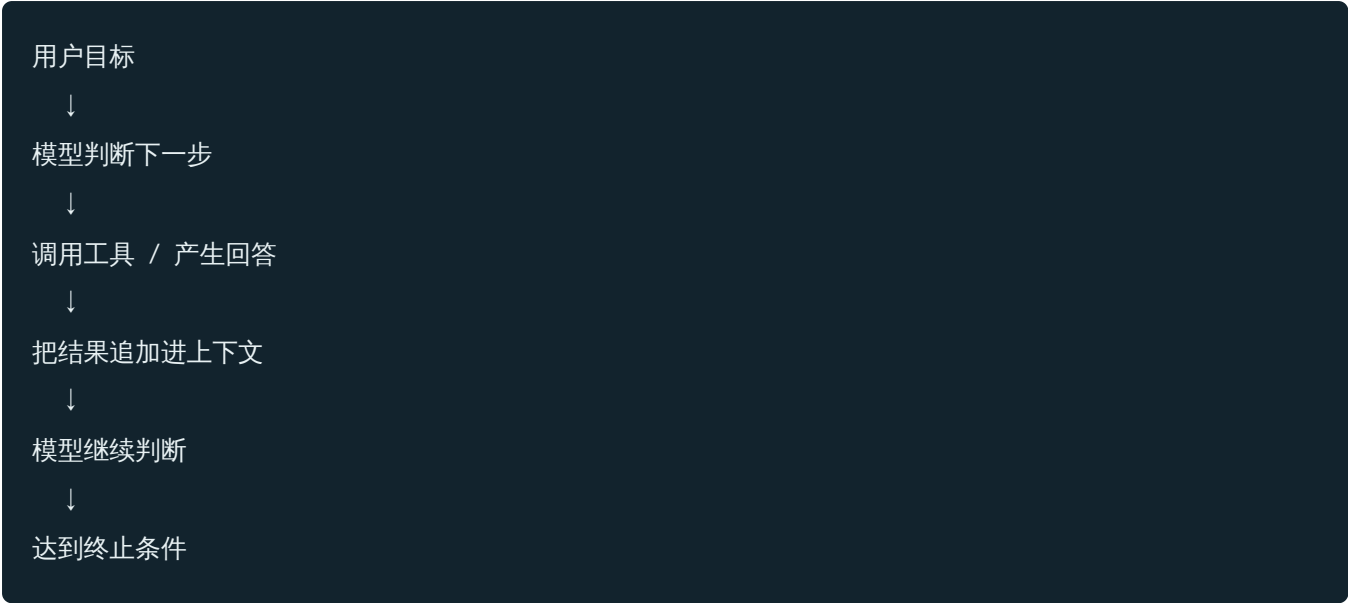
1. 解释一个可用 Agent 与“调用一次大模型 API”的本质区别；
2. 看懂 BrainPilot 从用户消息到多 Agent 协作、工具执行、事件流和持久化的完整链路；
3. 用 TypeScript、Node.js、Hono、React 和任意支持工具调用的模型 SDK，构建一个可运行的多 Agent 雏形；
4. 用明确的协议、权限、状态机、队列、可观测性和测试，使雏形可以稳健扩展；
5. 用 Codex 按小步、可验证的方式完成实现，而不是依赖一次性生成整个项目。

这不是 BrainPilot 的使用说明，而是一份“以源码为教材”的工程手册。你不需要先掌握多 Agent 理论，但应当会阅读 TypeScript、运行 npm 命令、理解 HTTP 和 React 基础。

1. 先建立正确心智模型

1.1 Agent 不是 Prompt，而是一个持续运行的闭环

最小 Agent 可以写成：



一个可工程化的 Agent 则至少包含七个部分：

部分	作用	BrainPilot 中的对应实现
模型适配	连接不同模型与服务商	<code>pi-provider.ts</code> 、 <code>provider-config.ts</code>
角色指令	界定目标、行为与交接协议	<code>personas.ts</code>
工具系统	让模型读写、检索、执行和通信	<code>tools/system-tools.ts</code> 、 <code>mcp-bridge.ts</code>
运行循环	驱动模型—工具—模型循环	Pi SDK + <code>agent-factory.ts</code>
状态权威	知道谁在运行、失败或停止	<code>SessionManager</code> 、 <code>MasAgent</code>
记忆与恢复	保存对话、邮箱、事件和产物	<code>persistent-layout.ts</code> 、JSON/JSONL 文件
可观测界面	流式显示文本、工具、状态和轨迹	AG-UI 事件、SSE、React reducers

所以，构建 Agent 的主要难点不是“写出更长的系统提示”，而是处理：

- 模型会失败、重复、沉默或偏离角色；
- 工具可能超时、返回巨量数据或造成破坏；
- 多个 Agent 会并发、互相等待、重复消费消息；
- 浏览器会断线并重连；
- 进程会重启，但未完成任务不能凭空消失；
- 协议会演进，前后端不能各自猜字段；
- 你需要知道系统为什么做出某个结果。

BrainPilot 的学习价值，正是在于它已经把这些问题变成了代码结构。

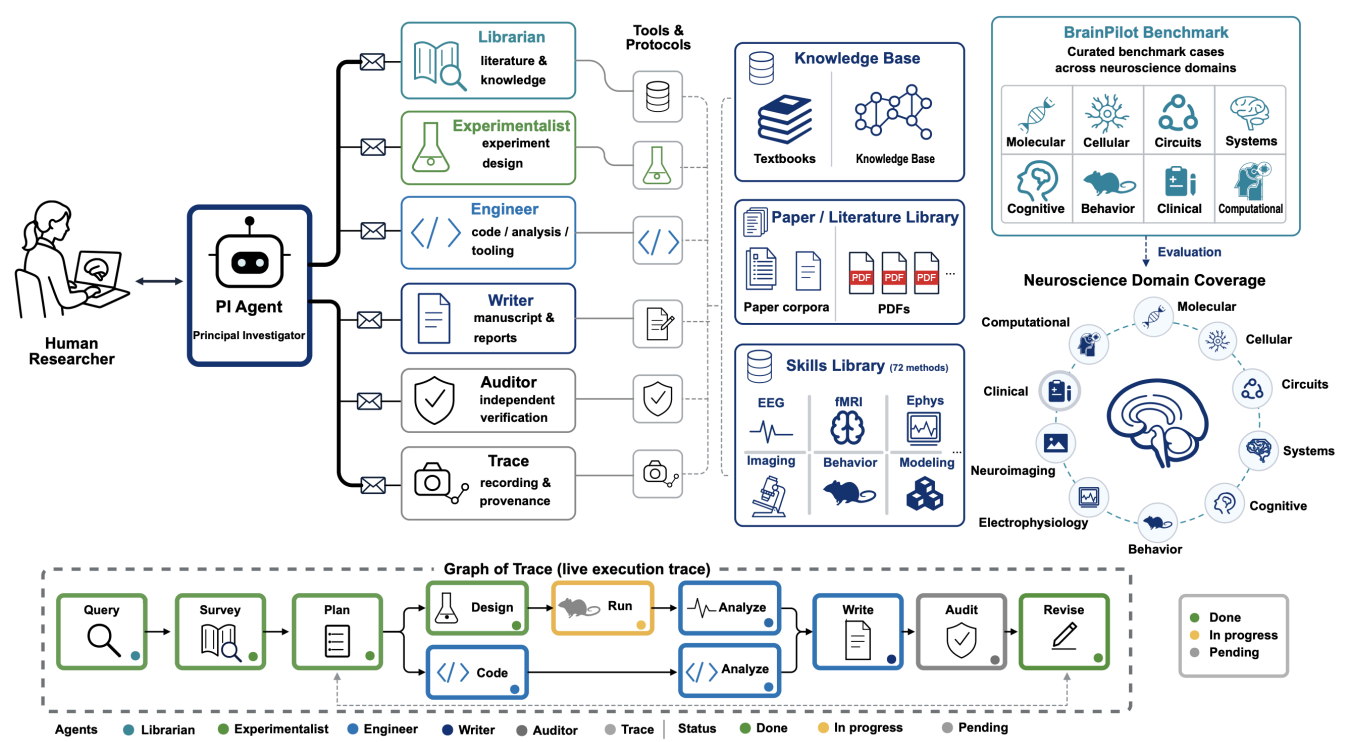
1.2 单 Agent、多 Agent 与 workflow

不要默认“Agent 越多越好”。

- 单 Agent：一个模型拥有若干工具，适合目标集中、上下文统一的任务。
- 多 Agent：不同角色拥有不同上下文、权限和认知职责，适合专业分工和并行工作。
- 确定性 workflow：代码明确规定每一步，适合必须稳定复现的业务流程。
- 混合架构：代码控制关键阶段，Agent 在阶段内部自主行动。通常最适合生产系统。

BrainPilot 是“主协调 Agent + 专家 Agent + 确定性运行时”的混合架构。主研究员负责理解用户、分解与综合；专家负责领域工作；运行时负责消息投递、权限、状态、持久化、并发和恢复。不要把这些基础设施责任交给模型“自行记住”。

2. 项目全景：先看边界，再看文件



2.1 Monorepo 分层

```
brainpilot/
├── packages/
│   ├── protocol/      # 跨进程、跨前后端的 Zod 协议单一事实源
│   ├── runtime/       # Agent 生命周期、工具、邮箱、轨迹、状态权威
│   ├── backend-core/  # 宿主 API、配置、runtime 编排与字节转发
│   ├── web/           # React 工作台、SSE 消费、消息/状态/轨迹展示
│   ├── skills/        # 可打包的领域技能内容
│   ├── cli/           # 安装、初始化、启动和进程管理
│   ├── client-cli/    # 面向 Runtime 的客户端/驱动
│   └── docs/          # 对外文档站
├── KnowledgeBase/     # 本地知识库构建脚本、向量存储与模型服务
├── docker/            # 主服务和沙箱镜像
└── scripts/           # 发布、冒烟测试和版本同步
```

分层背后的原则：

1. 协议不依赖 UI：`protocol` 只描述领域对象、HTTP 路由和事件。
2. Runtime 是状态权威：后端代理层不复制 Agent 状态，避免双写和漂移。
3. 后端负责宿主能力：服务商配置、Runtime 启停、静态资源和部署差异留在 `backend-core`。
4. 前端只折叠事件：React 不推测模型内部状态，而是消费快照与事件。
5. 领域知识与运行框架分离：技能和知识库可以增长，不必修改 Agent 核心循环。

这是你构建可扩展 Agent 时最应该复刻的部分。

2.2 技术栈

- Node.js ≥ 22 、TypeScript 5；
- npm workspaces；
- Pi Coding Agent SDK 作为模型与工具循环；
- Hono 作为 Runtime 和宿主后端；
- Zod 作为运行时协议校验；
- SSE 作为浏览器单向流式事件通道；
- React 18 + Vite 6；
- MCP SDK 作为外部工具桥；
- Vitest 作为测试框架；
- Python 只用于知识库构建与向量检索侧车。

2.3 推荐源码阅读顺序

按“契约 → 主流程 → 细节 → UI”阅读，比从入口文件漫游高效：

1. `packages/protocol/src/domain.ts`
2. `packages/protocol/src/events.ts`
3. `packages/protocol/src/http.ts`
4. `packages/runtime/src/types.ts`
5. `packages/runtime/src/session-manager.ts`
6. `packages/runtime/src/mas-agent.ts`
7. `packages/runtime/src/agent-factory.ts`
8. `packages/runtime/src/tools/system-tools.ts`
9. `packages/runtime/src/mailbox.ts`
10. `packages/runtime/src/trace.ts`
11. `packages/runtime/src/personas.ts`
12. `packages/runtime/src/server.ts`
13. `packages/backend-core/src/app.ts`
14. `packages/web/src/contexts/SSEContext.tsx`
15. `packages/web/src/contexts/SessionContext.tsx`
16. 前端各 reducer 与 Trace/Agent 视图

阅读每个文件时只回答四个问题：它拥有什么状态？接受什么输入？产生什么输出？失败后谁负责恢复？

3. 一条消息如何穿过整个系统

3.1 端到端时序



└─ 专家 prompt(envelopes)

└─ 结果再投递给 PI

所有事件 → EventBus → events.jsonl + SSE → React reducers → UI

3.2 为什么 HTTP 请求不等待 Agent 完成

`SessionManager.sendMessage()` 接受消息后启动异步 `agent.prompt()`，立即返回 `{ accepted, runId }`。长任务的输出通过 SSE 持续到达。

这样做有三个好处：

- HTTP 不会因为研究任务运行几十分钟而长期占用；
- 浏览器断线后可以重连，而任务继续运行；
- 一个统一事件流可以同时承载文本、工具调用、状态、轨迹和错误。

你的雏形也应采用“命令入口 + 事件出口”，不要让一个长 HTTP 请求承担整个 Agent 生命周期。

3.3 同一 Agent 收到并发消息

Pi 会阻止同一个 Agent 同时运行两个 `prompt()`。BrainPilot 检查 `agent.isStreaming`：

- 如果空闲，开启新 run；
- 如果正在运行，把新消息作为 `followUp` 注入当前 run；
- 不创建新的 run 生命周期，事件继续挂在当前 `runId` 下。

这是很实用的交互语义：用户可以在 Agent 工作时补充约束。更严格的业务系统也可以选择 `steer`、排队到下一 run，或拒绝并提示用户；关键是必须明确，而不是碰运气。

4. Runtime：整个系统最重要的层

4.1 SessionManager 为什么是核心

`SessionManager` 同时管理：

- session 的创建、恢复、更新、删除和驱逐；
- session 内的 Agent 注册表；
- Agent 状态与当前任务；
- 邮箱、Graph of Trace 和 EventBus；

- 服务商配置与工具开关；
- 用户输入请求；
- token、工具、技能和错误统计；
- 上传文件与工作区；
- 并发模型调用配额；
- 中断、恢复和内存保护。

这看起来职责很多，但它承担的是“一个 session 聚合根”的角色。关键不是一定要写成一个大类，而是要有且只有一个地方对运行状态负责。

反例：

```
前端认为 Agent 在运行
后端缓存认为 Agent 空闲
队列认为任务待处理
SDK 实际已经失败
```

BrainPilot 通过 Runtime 的 `SessionStateSnapshot` 把权威状态整体推送给前端，避免多个组件各自拼状态。

4.2 Agent 状态机



规范状态只有：

- `idle`
- `running`
- `error`
- `stopped`

同时记录：

- `activeRunId`
- 活跃工具调用 ID；
- 服务商重试状态；
- 最近错误与连续次数；
- 更新时间和存活标志。

状态改变时立即：

1. 更新 Runtime 内存；
2. 触发 `onStatusChange`；
3. 更新时间戳；
4. 推送新的 session 快照。

4.3 MasAgent：适配器与故障隔离层

`MasAgent` 不是另一个智能体框架，而是围绕一个 Pi `AgentSession` 的薄包装。它负责：

- 将 Pi 的事件翻译成统一 AG-UI 事件；
- 生成 run/message/tool call 标识；
- 管理本 Agent 权威状态；
- 累计真实 provider token 用量；
- 统计每 run 的工具、技能和错误；
- 捕获异常并输出 `RUN_ERROR`；
- 中断时等待原 prompt 完整退栈；
- 屏蔽 Pi 内部不需要暴露的事件。

这种“第三方 SDK 适配层”非常值得保留。未来更换模型 SDK 时，你只需重新实现 `IAgentSession` 和事件映射，不必改 UI、协议和 session 管理。

4.4 AgentFactory：把创建过程集中起来

真实 Agent 的创建集中在 `agent-factory.ts`：

1. 选择模型和服务商；
2. 设置 SDK 重试参数；
3. 把项目的 `SystemTool` 适配为 SDK 工具；
4. 装载角色 persona；
5. 禁用宿主机全局技能和上下文自动发现；
6. 只加入项目控制的技能路径；
7. 加载扩展钩子；
8. 配置允许的内置工具；

9. 打开该 Agent 的历史文件；
10. 返回统一 `IAgentSession`。

它特意设置 `noSkills: true` 与 `noContextFiles: true`，避免运行机器上的全局技能、`AGENTS.md` 或 `CLAUDE.md` 意外改变 Agent 身份。可复现性不只关乎模型版本，也关乎“到底注入了什么上下文”。

4.5 不要给长任务设置粗暴总超时

BrainPilot 的 Runtime 不为单次 turn 设置固定墙钟超时。任务终止来自：

- 正常完成；
- 明确错误；
- 用户主动中断；
- session 驱逐或关闭。

存活性依据是活动与真实状态，而不是“已经跑了多久”。托管层可以读取：

- `runningAgents`
- `lastActivityAt`

具体工具可以有独立超时，例如 shell 命令 30 分钟，但不要用一个 120 秒总超时杀死可能正常运行的研究任务。

5. 多 Agent 协作不是“互相聊天”，而是消息系统

5.1 角色设计

内置角色包括：

角色	主要职责
<code>principal</code>	理解用户、规划、委派、综合、决定最终输出
<code>librarian</code>	文献检索、知识综合、证据与空白识别
<code>experimentalist</code>	实验设计、变量操作化、控制与分析计划
<code>engineer</code>	代码、数据管线、执行、验证和产物
<code>writer</code>	报告组织、语言、格式和展示
<code>auditor</code>	检查数值、引用、文件声明、泄漏和证据链

角色	主要职责
trace	合并与编辑推理轨迹图

未知名称也可以创建，使用通用专家 persona。角色名称不是装饰，它同时决定：

- 系统提示；
- 系统工具；
- 内置文件/命令工具；
- 是否允许 MCP；
- 是否加载技能；
- UI 展示和总运行状态计算。

5.2 Persona 应写“契约”，不只写“性格”

一个稳健 persona 至少应包含：

1. 职责边界：负责什么，不负责什么；
2. 输入契约：会收到什么格式；
3. 输出契约：交付必须包含什么；
4. 工具策略：何时必须/不得调用工具；
5. 交接协议：任务完成后发给谁；
6. 证据规则：什么声明必须带证据；
7. 失败行为：无法完成时如何报告；
8. 终止条件：何时停止当前 turn。

BrainPilot 的专家被要求使用 `send_message(to="principal")` 交回结果，不能只在自身上下文里“说完”。这是协议，而不是礼貌用语。

5.3 邮箱模型

每条消息包含：

```
interface MailboxMessage {
    fromAgent: string;
    toAgent: string;
    content: string;
    msgType:
        | "user_message"
        | "result_deliver"
        | "task_delegate"
        | "trace_event"
        | "system";
    timestamp: number;
}
```

邮箱同时存在于内存和磁盘。写入会追加到目标 inbox，读取是原子 drain。未读消息可在进程重启后恢复。

当前保护：

- 每个 inbox 最多 20 条；
- 每次投递最多取 3 条；
- 每批正文最多约 24,000 字符；
- 第一条即使超限也会单独投递，避免永远卡住；
- 超容量写入明确失败，不静默丢弃。

这就是背压。没有背压的多 Agent 系统，很容易因为互相回复形成无限消息增长。

5.4 Delivery Loop

`send_message` 的过程不是直接调用目标 Agent：

1. 确保目标 Agent 存在；
2. 写入目标邮箱并持久化；
3. `wakeAgent(target)`；
4. 为 `sessionId:agentName` 注册投递循环；
5. 循环批量 drain 邮箱；
6. 将消息包装为带来源与类型的 `<message_envelope>`；
7. 调用目标 Agent；
8. 若运行期间又来消息，下一轮继续 drain；
9. 释放循环后再检查一次邮箱，消除“刚检查为空、消息随后写入”的竞态。

同一 Agent 的循环串行，不同专家可并行。session 级信号量默认最多允许 4 个并发 provider 调用，避免瞬间触发 429。

5.5 错误恢复与升级

专家运行错误时，Runtime 而不是 persona 负责基本恢复：

- 网络、限流、5xx 等可重试错误，在上限内重新投递给原专家；
- 当前实现最多处理 3 次 delivery 重试；
- 鉴权、配置等致命错误，或重试耗尽，升级给最近的实际委派者；
- 找不到委派者时回退给 principal；
- 专家成功交付后清除连续错误计数和委派者记录。

专家若没有报错、但忘记把结果交回，Pi 扩展会提醒一次；仍未交回时，宿主向真实委派者邮箱写入中性说明，避免永久死等。

这里的通用原则是：

Prompt 负责提高“通常会正确”的概率，Runtime 负责保证“失败后仍然可控”。

5.6 什么时候值得多 Agent

满足以下任意两项时再拆 Agent：

- 需要不同工具权限；
- 需要隔离大上下文；
- 工作可以真正并行；
- 有独立质量审查责任；
- 输出有稳定的专业交接格式；
- 某类任务需要独立模型或预算。

如果只是“让五个角色都发表意见”，通常会增加成本、延迟和冲突。

6. 工具系统与最小权限

6.1 三类工具

BrainPilot 的工具分为：

1. Pi 内置工具：`read`、`write`、`edit`、`bash`、`grep`、`find`、`glob`、`ls`；
2. 系统自定义工具：通信、Agent 管理、提问、轨迹、技能和本地知识库；
3. 外部 MCP 工具：运行时加载并适配外部服务。

6.2 角色工具矩阵

角色	文件/命令能力	编排能力	领域能力
principal	完整读写与 shell	通信、创建/销毁 Agent、提问、留痕	技能、知识库、MCP
engineer	完整读写与 shell	通信、留痕	技能、知识库、MCP
experimentalist	完整读写与 shell	通信、留痕	技能、知识库、MCP
writer	读写编辑，无 shell	通信、提问、留痕	技能、知识库、MCP
librarian	只读检索	通信、留痕	技能、知识库、MCP
auditor	只读检查 + 写审计报告；persona 禁止修改他人产物	通信、留痕	技能、知识库、MCP
trace	只读；无 MCP、无领域技能	专用图编辑工具	无

注意：权限不能只写在 prompt。模型可能误解或忽略文字，真正的限制必须通过“不向它注册该工具”实现。

6.3 工具定义要满足的条件

一个好工具应当：

- 名称表意清楚；
- 参数使用 JSON Schema/Zod；
- 描述说明副作用和返回；
- 参数在执行端再次校验；
- 错误以模型能识别的工具失败返回；
- 返回结果大小有上限；
- 具有明确授权边界；
- 尽可能幂等，或携带 idempotency key；
- 记录开始、结束、耗时和错误；
- 敏感参数不进入日志。

示例骨架：

```

type ToolResult = {
  content: Array<{ type: "text"; text: string }>;
  isError?: boolean;
};

interface SystemTool {
  name: string;
  description: string;
  parameters: Record<string, unknown>;
  execute(args: Record<string, unknown>): Promise<ToolResult>;
}

```

6.4 巨大工具结果

网页、日志、数据和检索结果可能一次塞爆上下文。BrainPilot 对工具结果统一包装：

1. 估算 token ；
2. 超限时截断 ；
3. 完整内容写入 workspace 文件 ；
4. 把文件路径与截断说明返回给 Agent ；
5. 发送用户可见警告。

这是一个可复用模式：大结果落盘，小摘要回上下文。

6.5 人在回路

`ask_user` 不是普通聊天，而是一个会阻塞当前 Agent 工具调用的异步请求：

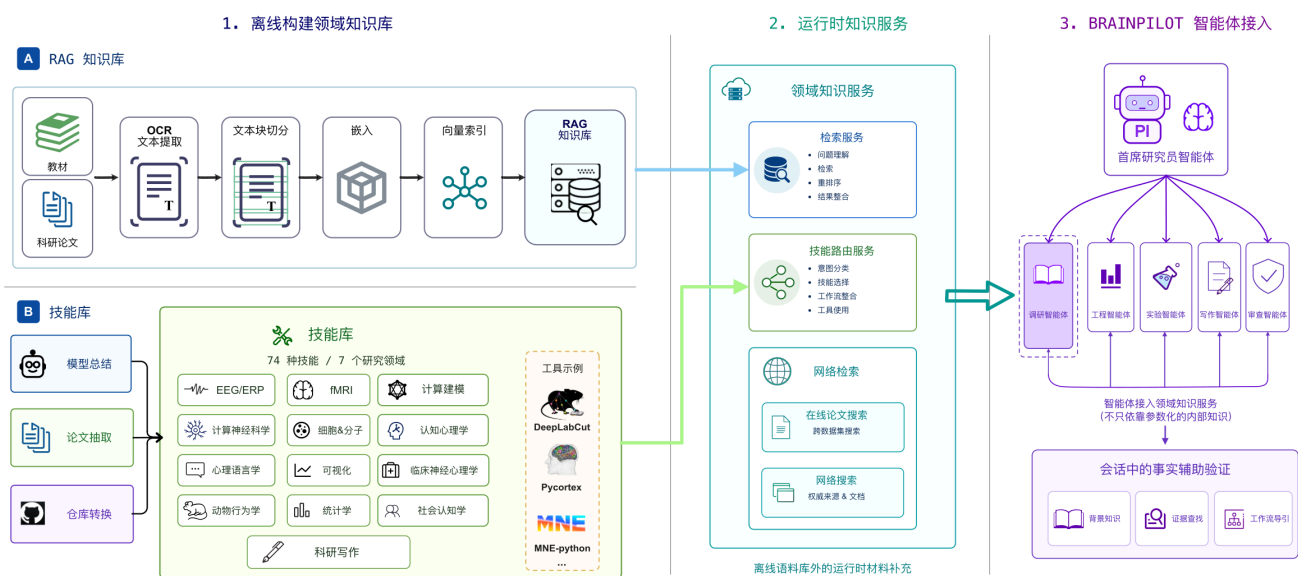
- 运行时持久化 `user_input_request` ；
- UI 展示卡片 ；
- 用户回答后持久化 `user_input_response` ；
- Deferred Promise 被解析，Agent 继续 ；
- 超时、删除、重启时必须取消并让 Agent 解除等待 ；
- 多个问题使用 FIFO，当前只展示队首 ；
- 当前上限为 8 个未决问题，回答最多 10,000 字符。

生产系统中，以下动作应优先要求人在回路：

- 外部发送消息、发布、付款、删除 ；
- 修改权限或凭据 ；
- 高影响分析结论 ；

- 目标、受众或成功标准存在关键歧义；
- 模型需要扩大任务范围。

7. 技能、知识库与上下文工程



7.1 Prompt、Skill、Tool、Knowledge 的区别

概念	回答的问题	例子
Persona/Prompt	“你是谁，如何工作？”	principal 的委派规则
Skill	“完成某类任务的标准方法是什么？”	EEG 预处理指南
Tool	“你能执行什么动作？”	搜索论文、运行代码
Knowledge	“此领域有哪些可检索事实？”	本地论文向量库
Memory	“这个用户/session 之前发生了什么？”	对话历史、共享文件

不要把它们全塞进一个系统提示。它们有不同生命周期、权限和更新频率。

7.2 渐进式披露

仓库内约有 74 个 `SKILL.md`。如果全部正文注入每个 Agent，会浪费上下文并干扰选择。

BrainPilot 把技能分成两层：

- `01_Meta-Skills` 物化到 `bp_template/skills/`，作为高频 always-on 技能；只有名称和描述先进入上下文，正文按需读取；
- 其他领域技能物化到 `bp_template/skills-router/`，Pi 完全看不到；Agent 先用 `skill_search` 按关键词查找，再按名称加载全文。

部署时复制遵循“存在则不覆盖”，用户修改可以跨升级保留。

7.3 一个技能的建议结构

```
category/
├── skill-name/
│   ├── SKILL.md
│   ├── references/      # 需要时下钻
│   ├── scripts/        # 可重复执行的确定性步骤
│   └── assets/          # 模板或资源
```

`SKILL.md` 至少应说明：

- 何时使用、何时不用；
- 所需输入；
- 步骤；
- 输出格式；
- 验证方法；
- 常见失败；
- 需要读取的 references 或执行的 scripts。

高质量 Skill 的价值不在“告诉模型更多知识”，而在把团队经验固化为可重复流程。

7.4 本地知识库

`KnowledgeBase/` 提供：

- PDF/OCR、元数据抽取和切块；
- 向量化与向量存储；
- 本地模型侧车服务；
- 论文检索与领域知识检索工具。

知识库负责召回事实，Skill 负责指导如何用这些事实。RAG 返回了十段文本，不代表 Agent 会做出正确的方法学判断；因此两者应组合。

7.5 上下文预算

建议给每次模型调用设置预算表：

系统与工具描述	15%
角色与技能摘要	10%
最近对话	25%
任务相关检索	25%
工具返回	15%
输出余量	10%

比例不是固定标准，但你应当能回答“为什么这段内容必须进入本次上下文”。长期系统还需要：

- 旧消息摘要；
- 工具结果落盘；
- 按任务检索历史；
- 对稳定事实与临时过程分层；
- 在摘要中保留未决事项、约束和证据指针。

8. 协议、事件流与前端

8.1 Protocol 是单一事实源

@brainpilot/protocol 用 Zod 定义：

- Session 与 Agent 状态；
- provider 配置；
- token 与统计；
- Graph of Trace；
- HTTP 请求/响应；
- AG-UI 事件联合类型。

跨边界数据不要只写 TypeScript interface。interface 在运行时消失，无法保护你免受旧客户端、坏数据和手写请求。建议采用：

Zod Schema → 推导 TypeScript 类型 → 生成/共享协议 → 边界解析

8.2 事件类型

主要事件包括：

- 文本：`TEXT_MESSAGE_START/CONTENT/END/CHUNK`
- 工具：`TOOL_CALL_START/ARGS/END/RESULT`
- 推理：`REASONING_*`
- 运行：`RUN_STARTED/FINISHED/ERROR`
- 快照：`MESSAGES_SNAPSHOT`、`STATE_SNAPSHOT`
- 自定义：`CUSTOM`
- 人类输入：`user_input_request/response/cancelled`
- 系统消息：`system_message`

线上的字段保留 snake_case，Web 在边界统一规范化为 camelCase。字段命名转换只能在一个明确位置做，不能散落在组件中。

8.3 EventBus 的三重职责

每个 session 有独立 EventBus：

1. 发布给当前订阅者；
2. 保留最近事件环形缓冲区，供 SSE 新订阅者快速重放；
3. 追加到 `events.jsonl`，供完整历史恢复。

环形缓冲区默认 500 条，只解决短时断线；完整对话恢复从持久化历史读取，前端再按消息 ID 与 live tail 去重合并。

对于“用户回答”这类生命周期关键事件，使用 `emitDurable`：先成功落盘，再发布和确认。普通流式 token 可以 best-effort 落盘。不是所有事件都需要同一持久化等级。

8.4 SSE 重连

前端 `SSEContext` 实现：

- 每个 session 一条 EventSource；
- 连接状态：`idle / connecting / open / error`；
- 指数退避重连；
- open watchdog，避免浏览器永远停在 connecting；
- bfcache、页面恢复和 tab 重新可见时检查僵死连接；
- 心跳过滤；
- 解析失败转成可见错误事件；
- 每个 session 独立事件队列。

SSE 适合服务端到浏览器的持续推送，简单且能自动携带浏览器鉴权语义。若需要高频双向实时协作，再考虑 WebSocket。

8.5 Reducer 思维

React 不直接“处理所有逻辑”，而是把事件分别折叠为：

- `messages`
- `agents`
- `trace`
- `tokenUsage`
- `runActive`

这相当于事件溯源的读模型。测试一个 reducer 比测试整个页面容易得多，也使历史重放和实时事件使用同一套逻辑。

9. 持久化、恢复和工作区

9.1 数据分层

```
<BP_DATA_DIR>/
├── data/                # 跨 session 持久文件
├── workspaces/<sessionId>/ # 单 session 工作区与产物
└── bp_template/
    ├── agents/<name>/prompt.md # 用户可编辑 persona
    ├── skills/                # always-on 技能
    ├── skills-router/         # 路由技能
    └── tool_toggles.json
```

每个 Agent 还有独立历史，session 内有：

- `events.jsonl`
- 邮箱 JSON
- `trace.json`
- token usage
- run stats
- session 元数据

9.2 单用户信任边界

Runtime 的数据根是单用户的。多用户托管必须为每个用户提供独立 `BP_DATA_DIR` 或 volume，不能只在同一个数据目录里加 `userId` 子目录并假装隔离。

本地非 Docker 模式下，Agent 直接在宿主机运行，没有容器沙箱。适合可信个人环境，不适合把不可信用户和强力 shell 工具放在一起。

9.3 恢复策略

至少区分四类状态：

状态	推荐恢复方式
对话与事件	追加日志重放
Agent 历史	SDK session history 恢复
未读 Agent 消息	mailbox 文件恢复并重新唤醒
正在等待用户的问题	旧 Promise 不可恢复，应写取消终态并让新任务重新决策

不要试图把 JavaScript Promise 序列化。恢复的是业务事实，不是进程内控制流。

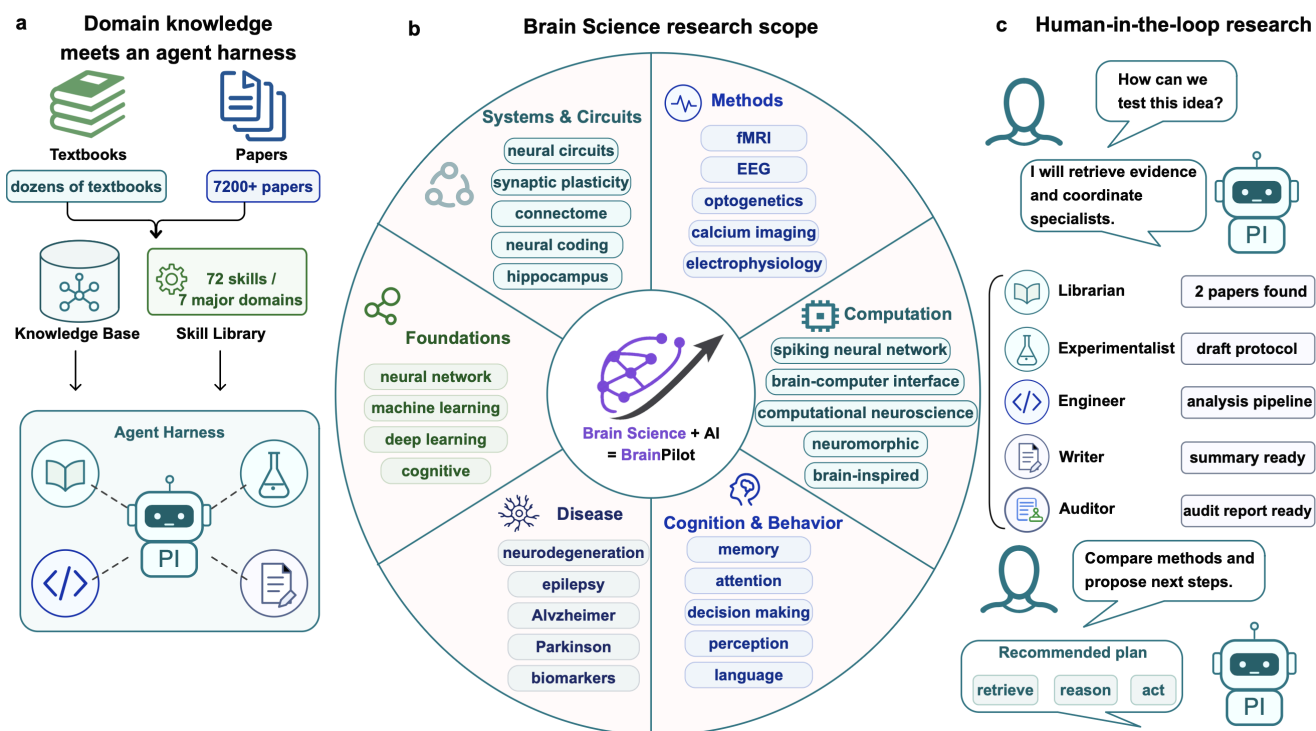
9.4 写入一致性

文件型雏形足够学习，但仍要做到：

- 同一文件写入串行化；
- 关键事件先落盘再确认；
- append-only 日志便于恢复；
- 读取损坏文件时有清晰降级；
- 更新快照时避免部分写入，成熟后采用临时文件 + 原子 rename；
- 所有任务、消息和运行有稳定 ID。

当系统进入多进程、多机器阶段，再把 mailbox/job/state 迁移到 Postgres、Redis Streams、NATS 或专用队列。

10. Graph of Trace : 把“过程”变成产品能力



10.1 它记录什么

Graph of Trace (GoT) 是一个推理/工作 DAG。节点可以包含：

- 标题、类型与状态；
- 创建它的 Agent；
- 描述、摘要和正文；
- 父节点与关系；
- 产物路径；
- 时间戳和元数据。

边表示“前置工作 → 依赖它的后续工作”。代码对 `depends_on` 做时间顺序保护：如果模型把方向写反，会依据创建时间交换端点。

10.2 为什么单独设置 Trace Agent

普通 Agent 调用 `record_trace` 时并不直接修改图，而是给 Trace Agent 写一条 `trace_event`。Trace Agent 再判断：

- 新建节点；
- 更新已有节点；
- 合并重复节点；

- 添加关系。

好处是图的编辑责任集中，并且 Trace Agent 本身的运行状态可见。代价是多一次模型调用。对于成本敏感系统，可以把常规节点改为确定性记录，只把去重和摘要交给模型。

10.3 轨迹不是完整思维链

产品应该记录：

- 做了什么决定；
- 使用了什么证据；
- 调用了什么工具；
- 产生了什么产物；
- 哪一步依赖哪一步；
- 有什么风险和未决问题。

不需要也不应要求模型暴露私密、逐 token 的内部思维过程。可审计的行动与证据比“完整心理独白”更可靠。

11. 可靠性与安全：从雏形开始就要有

11.1 BrainPilot 已体现的可靠性模式

- 单一状态权威；
- 每 Agent 异常隔离；
- provider 指数退避重试；
- 专家 delivery 重试与升级；
- 邮箱容量和批大小背压；
- session 级并发上限；
- 工具结果截断与落盘；
- 事件环形缓冲 + 完整持久化历史；
- SSE watchdog 与重连；
- 中断时清空待投递消息，避免 Agent 被重新唤醒；
- 内存软阈值拒绝新任务；
- 工具白名单；
- 宿主上下文和技能显式装载；
- auditor 对数值、引用、产物声明与分析流程做交付前审查；
- 111 个测试文件覆盖协议、Runtime、后端、CLI 和 Web 关键路径。

11.2 你需要补强的生产能力

如果要做“类似甚至更好”的系统，优先补：

- 1. Durable Job Ledger：每个用户任务有 `queued/running/waiting_user/succeeded/failed/cancelled` 状态，而不只依赖 Agent 状态；
- 2. 幂等消费：message/job 携带唯一 ID，收件方记录已处理 ID；
- 3. 结构化交接：专家返回 Zod 校验对象，而非完全自由文本；
- 4. 权限策略层：工具调用前根据用户、角色、资源、风险和环境作决定；
- 5. 审批与补偿：高风险写操作先申请批准，失败后有撤销或补偿动作；
- 6. 预算控制：每任务 token、金额、时间、并发和工具额度；
- 7. OpenTelemetry：把 run、模型调用、工具调用、队列等待统一为 trace/span；
- 8. 评测基线：任务成功率、事实正确性、工具成功率、延迟和成本可回归；
- 9. 确定性编排：对必须按顺序发生的步骤用代码状态机，而非只用 persona；
- 10. 数据治理：租户隔离、密钥保险库、日志脱敏、保留周期、导出和删除。

11.3 常见失败模式

失败	表现	修复方向
Prompt 包办一切	偶发漏步骤、无法恢复	把状态、重试、权限放到代码
共享一个超长上下文	干扰、成本上涨	按 Agent/任务隔离，结构化交接
无状态权威	UI 与后端不一致	Runtime 推权威快照
无消息背压	Agent 互相回复爆炸	inbox 上限、批处理、并发限制
工具权限过大	搜索角色也能删文件	按角色只注册必要工具
全量技能注入	系统 prompt 膨胀	元数据预览 + 按需加载
只存最终答案	无法恢复、无法审计	事件日志 + 产物 + 轨迹
只测 happy path	线上卡死	测重连、重复、超限、中断、恢复
盲目自动重试	重复副作用	只重试可重试错误，写操作幂等
多 Agent 只为“显得高级”	慢、贵、意见冲突	按权限/上下文/并行性决定拆分

11.4 许可证

仓库使用 AGPL-3.0-only。学习、修改和构建衍生系统前，应理解其网络服务场景下的源代码提供义务。若你的目标是闭源商业产品，不要直接复制受 AGPL 约束的代码；可以学习架构思想，进行独立实现，并在需要时咨询专业法律意见。

12. 从零构建一个稳健雏形

下面不是复刻所有功能，而是按风险优先级构建“可生长内核”。

阶段 0：定义一个窄用例

先选一个明确任务，例如：

用户上传一组项目文件，Coordinator 制定修改计划，Engineer 实现，Reviewer 检查，最终给出变更摘要。

写清：

- 用户输入；
- 成功产物；
- 允许工具；
- 不允许行为；
- 哪些决策需要用户确认；
- 最大成本与等待时间；
- 如何判断成功。

不要以“做一个通用 Agent 平台”为第一目标。

阶段 1：协议优先

建议目录：

```
packages/  
├── protocol/  
├── runtime/  
├── server/  
└── web/
```

先定义：

```

const AgentStatusSchema = z.enum(["idle", "running", "error", "stopped"]);

const RunSchema = z.object({
  id: z.string(),
  sessionId: z.string(),
  agent: z.string(),
  status: z.enum([
    "queued",
    "running",
    "waiting_user",
    "succeeded",
    "failed",
    "cancelled",
  ]),
  createdAt: z.string(),
  updatedAt: z.string(),
});

const EventSchema = z.discriminatedUnion("type", [
  z.object({ type: z.literal("run.started"), runId: z.string() }),
  z.object({ type: z.literal("text.delta"), runId: z.string(), delta: z.string() }),
  z.object({
    type: z.literal("tool.finished"),
    runId: z.string(),
    toolCallId: z.string(),
    ok: z.boolean(),
  }),
  z.object({ type: z.literal("run.finished"), runId: z.string() }),
  z.object({ type: z.literal("run.failed"), runId: z.string(), error: z.string() }),
]);

```

验收：协议包有类型检查和序列化/反序列化测试。

阶段 2：只做一个 Agent

实现统一接口：

```
interface AgentSession {
  readonly isStreaming: boolean;
  subscribe(listener: (event: ModelEvent) => void): () => void;
  prompt(input: string): Promise<void>;
  followUp(input: string): Promise<void>;
  abort(): Promise<void>;
  dispose(): void;
}
```

再做：

- `AgentFactory`
- `AgentAdapter`
- `SessionManager`
- mock model/session

验收：

- 输入“hello”可以流式输出；
- 失败会进入 `error` 并产生终态事件；
- abort 后一定产生终态；
- mock 测试不需要真实 API。

阶段 3：工具与权限

先实现三个工具：

- `read_file`
- `write_file`
- `ask_user`

加入：

- schema 校验；
- workspace 路径规范化；
- 禁止 `..` 越界；
- 结果大小限制；
- 工具事件；
- 角色白名单。

验收：只读角色无法看到 `write_file`，不是调用后才被 prompt 劝退。

阶段 4：持久事件流

实现：

- 每 session 一个 EventBus ；
- SSE ；
- 最近事件缓冲 ；
- `events.jsonl` ；
- 完整历史 API ；
- 前端 reducer ；
- 重连和去重。

验收：刷新浏览器后历史不丢；断网再恢复时不重复消息；坏事件不让 UI 崩溃。

阶段 5：第二个 Agent 与邮箱

只加入 Coordinator 和 Engineer：

- `send_message` ；
- 每 Agent 独立 inbox ；
- 写入后主动 wake ；
- 同 Agent 串行 delivery loop ；
- inbox 上限、批处理 ；
- 消息 ID 与幂等处理 ；
- session 并发上限。

验收：Coordinator 能委派，Engineer 能交回，进程重启后未读消息仍可处理。

阶段 6：恢复与失败策略

加入：

- 错误分类：retryable / fatal / policy / user ；
- 有界指数退避 ；
- 连续失败计数 ；
- 升级给委派者 ；
- 无回复 fallback ；
- 中断清队列 ；
- waiting_user 的恢复取消。

验收：用测试注入 429、401、断线、进程重启、工具超限和专家沉默。

阶段 7：Skill 与检索

先放 3 个真正有用的 Skill：

- 计划；
- 实现；
- 代码审查。

实现元数据搜索和按需全文加载，再增加 RAG。不要反过来先建庞大知识库。

验收：可以记录“哪个 run 查询/加载了哪个 skill”，并比较启用前后的任务质量。

阶段 8：审查与评测

加入 Reviewer/Auditor，但要给它：

- 原始用户目标；
- 结构化交接包；
- 草稿/产物路径；
- 代码与测试证据；
- 明确审查 rubric。

建立至少 20 个固定任务，记录：

任务成功率
工具调用成功率
重复/越权调用率
平均首 token 延迟
端到端时长
输入/输出 token
估算成本
需要人工纠正次数
恢复成功率

只有有基线，才能判断“更好”。

13. 用 Codex 进行 vibe coding 的正确方式

13.1 Codex 最适合的工作单元

把每次请求限制为一个可验证的垂直切片：

协议 + 最小实现 + 单元测试 + 必要文档

不推荐：

帮我做一个比 BrainPilot 更好的 Agent 平台。

推荐：

阅读 `packages/protocol` 和 `packages/runtime/src/event-bus.ts`。在我的新项目中实现一个最小的、每 session 隔离的事件总线：支持订阅、500 条环形缓冲、JSONL 串行持久化和 durable emit。先写 Zod 事件协议与失败测试，再实现。不要改 UI。完成后运行相关测试并解释持久化顺序。

13.2 每次给 Codex 的任务模板

目标：

[一个可观察结果]

允许范围：

[允许修改的目录/文件]

上下文：

[参考文件、现有约束、为何要改]

不做：

[明确排除项]

验收标准：

1. [...]

2. [...]

3. [...]

验证命令：

[typecheck/test/build]

工程要求：

- 先检查现有实现与测试
- 保持协议单一事实源
- 不绕过类型
- 失败路径必须测试
- 报告剩余风险

13.3 推荐的 Codex 迭代循环

1. Inspect : 让 Codex 先读代码并画出当前数据流
2. Specify : 让它把需求写成可测试契约
3. Implement : 只做一个切片
4. Verify : 运行定向测试、typecheck、build
5. Review : 检查 diff、状态所有权、失败路径和越权
6. Commit : 小提交, 写清“为什么”
7. Expand : 再进入下一个切片

你的作用不是逐行替 Codex 写代码, 而是守住架构边界和验收标准。

13.4 可直接复制的学习 Prompt

Prompt A : 理解一个模块

请把 ``packages/runtime/src/mailbox.ts`` 当作教学源码阅读。

输出 :

1. 它拥有的状态 ;
2. 每个 public 方法的前置条件、后置条件和失败 ;
3. 它解决的竞态与背压问题 ;
4. 哪些保证只适用于单进程 ;
5. 如果迁移到 Postgres, 协议和调用方应如何保持不变。

只分析, 不修改代码。所有结论引用具体符号名。

Prompt B : 实现最小版本

在 ``packages/runtime`` 实现 Mailbox v1 :

- 每 Agent 独立 FIFO ;
- write/readBatch/count ;
- 单 inbox 上限 20 ;
- 批量最多 3 条或 24000 字符 ;
- JSON 文件持久化 ;
- 同一文件写入串行 ;
- 重启 recover ;
- 测试边界、顺序、超限和损坏文件。

先给出接口与测试计划, 然后直接实现并运行测试。

Prompt C：做可靠性审查

审查这次 diff，不修改代码。重点找：

- 双重状态权威；
- 消息丢失或重复消费；
- 缺少终态事件；
- 同 Agent 并发 prompt；
- 重试导致重复副作用；
- 路径越界；
- 敏感信息进入日志；
- 浏览器重连后的重复 UI。

按严重程度列出文件、符号、触发步骤和最小修复建议。

Prompt D：加入新角色

加入 `reviewer` Agent，但不要只加 persona。

要求：

- 在协议/registry 中明确角色；
- 只能读 workspace，不可 edit/write/bash；
- 可 send_message 和 record_trace；
- 输入必须是结构化 review packet；
- 输出通过 Zod 校验；
- 未交回时有 fallback；
- 有权限与交接测试；
- 更新架构文档。

先检查角色、工具配置、ensureAgent 和 delivery loop 的现有扩展点。

13.5 你必须亲自做的判断

不要把这些决策完全交给 Codex：

- 产品的成功标准；
- 哪些动作高风险；
- 数据与租户信任边界；
- 预算；
- 角色是否真的需要拆分；
- 哪些步骤必须确定性；
- 什么证据足以对用户作出声明；

- 许可证和合规选择。

Codex 能极大提高实现速度，但“更快地产生代码”不等于“更快地产生正确系统”。

14. 建议练习：从阅读到能独立构建

练习 1：手动画出状态所有权

从源码填表：

状态	唯一写入者	持久化位置	前端如何获得
Agent status			
runActive			
mailbox			
messages			
trace			
provider config			

参考答案方向：Agent/运行状态由 Runtime 写；provider 本地配置由 backend-core 写；前端通过快照、事件和配置 API 获得。

练习 2：增加一个无副作用工具

增加 `workspace_summary`：

- 只列文件名、大小和更新时间；
- 禁止越界；
- 对大目录分页；
- librarian 可用，trace 不可用；
- 有 TOOL_CALL 事件；
- 有权限和结果截断测试。

练习 3：制造故障

用 mock Agent 依次模拟：

- 第二次 provider 调用 429 ；
- 专家运行成功但不 `send_message` ；
- mailbox 已满 ；
- SSE 在消息中间断线 ；
- `events.jsonl` 最后一行损坏 ；
- 用户在 ask_user 时删除 session。

写出你期望看到的最终状态，再让 Codex 补测试。

练习 4：结构化交接

把 Engineer 的自由文本交接替换为：

```
const EngineerHandoffSchema = z.object({
  summary: z.string(),
  changedFiles: z.array(z.string()),
  commandsRun: z.array(z.string()),
  testResults: z.array(z.object({
    command: z.string(),
    passed: z.boolean(),
    outputPath: z.string().optional(),
  })),
  risks: z.array(z.string()),
  nextActions: z.array(z.string()),
});
```

允许正文作为展示，但 Runtime 只把校验成功的对象标记为正式交付。

练习 5：做一次评测对比

选 10 个固定开发任务，对比：

- 单 Agent ；
- Coordinator + Engineer ；
- Coordinator + Engineer + Reviewer。

记录成功率、成本、耗时和人工纠正次数。你很可能会发现：第三个 Agent 只在高风险任务上划算。这就是评测驱动架构，而不是审美驱动架构。

15. 如何构建“比蓝本更好”的下一代雏形

15.1 推荐目标架构



相较当前 BrainPilot，最大的升级是把“用户任务/ workflow 状态”提升为一等对象。Agent 的 `running` 只是执行细节；真正需要恢复和运营的是：

- 用户想完成什么；
- 当前处于哪个业务阶段；
- 等待谁；
- 已花费多少；
- 哪些步骤可重试；
- 哪些副作用已发生；
- 最终验收是否通过。

15.2 自主程度分级

为每类任务标记自治等级：

等级	行为
L0	只回答，不调用工具
L1	只读工具，无副作用
L2	在隔离 workspace 内写文件和运行代码
L3	修改外部系统前逐次批准
L4	在预算和策略内自动执行可补偿外部动作
L5	高自治长期任务，必须有强审计和运营机制

默认从 L1/L2 起步。不要因为模型“看起来很聪明”就跳到 L4。

15.3 用确定性代码包住概率模型

适合交给模型：

- 理解自然语言；
- 提出计划；
- 选择相关 Skill；
- 综合证据；
- 生成草稿；
- 在约束内选择工具。

适合代码强制：

- 权限；
- schema；
- 阶段顺序；
- 幂等；
- 重试上限；
- 预算；
- 审批；
- 持久化；
- 终态；
- 删除和外部副作用。

最稳健的 Agent 系统，不是模型最自由，而是自由发生在设计好的边界内。

16. 30 天学习与构建路线

第 1 周：读懂

- Day 1：读 `protocol`，画出领域对象和事件表；
- Day 2：读 `SessionManager.sendMessage()` 和 `MasAgent.prompt()`；
- Day 3：读 tools、权限矩阵和 persona；
- Day 4：读 mailbox 与 delivery loop；
- Day 5：读 EventBus、SSE 和 React reducers；
- Day 6：读持久化、恢复、中断和错误分类；
- Day 7：用文字复述完整时序，不看源码补图。

交付：一张自己的架构图、一张状态所有权表、十个仍不理解的问题。

第 2 周：单 Agent 内核

- 初始化 TypeScript monorepo；
- 完成 protocol；
- mock Agent；
- 一个真实模型 adapter；
- SessionManager；
- EventBus + SSE；
- 三个安全工具；
- React 最小聊天页。

交付：刷新可恢复、失败有终态、可主动停止的单 Agent。

第 3 周：多 Agent 与恢复

- Coordinator + Engineer；
- mailbox；
- delivery loop；
- 背压和并发；
- 错误分类、重试与升级；

- ask_user ;
- 结构化交接。

交付：可在重启后继续处理未读委派的双 Agent 系统。

第 4 周：质量与扩展

- Skill router ;
- Reviewer ;
- 20 个评测任务；
- token/成本/延迟统计；
- 安全检查；
- Docker 隔离；
- 写扩展指南。

交付：一个有基线、有权限、有恢复测试、能新增角色和工具的稳健雏形。

每天让 Codex 做一个垂直切片，每天结束必须有可运行测试。不要积累三天未验证代码。

17. 完成定义 (Definition of Done)

当你的项目满足下面清单时，才可以称为“可稳健扩展的雏形”。

架构

- [] 协议包是跨层类型单一事实源；
- [] Runtime 是 Agent/Run 状态唯一权威；
- [] 模型 SDK 被 adapter 隔离；
- [] Persona、Skill、Tool、Knowledge、Memory 分层；
- [] 新角色和新工具有明确注册点。

运行

- [] 每个 run 有唯一 ID 和明确终态；
- [] 同一 Agent 不会并发 prompt；
- [] 多 Agent 消息有队列、背压与幂等；
- [] 可重试错误和致命错误分类；
- [] retry 有次数、退避和预算上限；

- [] 用户可中断，且中断后不会被旧消息复活；
- [] 需要人的决策可以持久等待并正确取消。

安全

- [] 工具按角色最小授权；
- [] 路径被规范化并限制在 workspace；
- [] 高风险副作用需要批准；
- [] 密钥不进入 prompt、事件和日志；
- [] 多用户数据有真正隔离；
- [] 本地无沙箱模式向用户明确提示。

可恢复性

- [] 对话和关键事件持久化；
- [] 浏览器断线重连不重复；
- [] 进程重启后能恢复 session 与未读工作；
- [] waiting_user 等不可恢复控制流有明确关闭策略；
- [] 损坏尾日志能降级或修复。

可观测性

- [] 可查看 Agent 状态、run、工具调用和错误；
- [] 可追踪产物与证据；
- [] 统计 token、成本、延迟和失败；
- [] 有任务级轨迹，而不是只看最终回答；
- [] 日志可以按 session/run/toolCall 关联。

质量

- [] mock 模式无需 API；
- [] 协议、状态机、队列、重试、恢复和权限有测试；
- [] 有固定评测集与基线；
- [] 每次模型、prompt、skill 或工具变更可回归；
- [] README 能让新开发者在 30 分钟内跑起来。

18. 术语速查

术语	简明解释
Agent loop	模型决策、调用工具、读取结果并继续的循环
Persona	角色职责与行为契约
Tool calling	模型输出结构化工具请求，由宿主执行
MCP	把外部工具/资源以标准协议提供给模型
Skill	可按需加载的方法、流程和模板
RAG	从外部知识中检索与当前问题相关内容
Session	一段独立对话与工作空间
Run	某 Agent 对一次输入的完整处理生命周期
Event sourcing	通过追加事件重建读模型与历史
SSE	服务端持续向浏览器推送事件的 HTTP 通道
Backpressure	下游处理不过来时限制上游继续堆积
Idempotency	同一请求重复执行不会产生重复副作用
State authority	对某状态拥有最终解释权的唯一组件
HITL	Human in the Loop，关键步骤由人参与决策
GoT	Graph of Trace，可检查的工作/推理依赖图
Handoff	Agent 之间的结构化任务或结果交接
Durable	进程重启后仍然存在

结语

从 BrainPilot 最应该学走的，不是脑科学角色名称，也不是某一段长 prompt，而是以下组合：

- 1. 协议先行；
- 2. Runtime 拥有状态；

3. SDK 通过 adapter 隔离；
4. 多 Agent 通过持久消息交接；
5. 权限由工具注册强制；
6. 技能按需披露；
7. 事件实时推送并可重放；
8. 失败有重试、升级和终态；
9. 过程、证据和产物可观察；
10. 用评测证明改进。

先构建一个可靠的 Coordinator + Engineer，再增加 Reviewer、Skill、RAG 和图形化轨迹。每增加一层智能，都同时增加对应的协议、边界、失败测试和可观测性。这样你用 Codex 获得的就不只是一个演示，而是一块真正可以继续生长的 Agent 内核。